

Reproducing the first question proof of Erdős 883

5 October 2026

This guide separates artifact identification, mathematical inspection, a full local-source replay, and the limits of the existing audit. It accompanies the formal-source-aligned manuscript. The primary result is `Erdos883Verified.erdos883_firstQuestion`, the all- n first question only.

1 Identify the exact proof artifact

Use `Erdos883_FirstQuestion_Lean_20261005.zip`, version 5. Its size is 20359760 bytes. The SHA-256 is the concatenation of

```
8f017032cb93f2f0891f6f8200390c161aca
```

```
bf112dac72567462e642b91ede95
```

Run `sha256sum` on Linux, or `shasum -a 256` on macOS, against the archive before extraction. An unequal hash means it is a different artifact. Extract into a new directory and work from the `erdos883-first-question` directory. Preserve the original archive and a read-only extracted copy if collecting independent replay evidence.

The source archive has no required local compiled proof objects. It includes 6831 local source modules, the dependency order in `MODULES.txt`, the manifest `PROVENANCE.json`, final audit documents, and the checking harness. Read its `README.md` and `VERIFICATION.md`. Historical mutable-workspace status documents are not a substitute for those files inside the identified archive.

2 Inspect the mathematical statement first

Read `CanonicalTarget.lean`, `Erdos883CycleLengths.lean`, and `VERIFY_CANONICAL.lean`. Check that the density threshold uses natural-number division, the set is contained in $[n]$, the graph joins distinct coprime vertices, and the cycle definition is the intended simple-cycle notion. The raw harness expands the graph alias and types the final theorem at the entire universally quantified proposition.

The canonical reference is the right-hand side of part (i) in the Formal Conjectures source at commit `89294ea02bd7cd678d59984add52cb4baef3dbf4`. The second question is outside the claim. Small n is not omitted: $n < 6$ is vacuous, the finite prefix reaches 199999, and the analytic theorem starts at 200000.

3 Prepare the pinned environment

Use the official Lean toolchain selected by `lean-toolchain: leanprover/lean4:v4.33.1`. Use Lake's checked-in configuration and lockfile to fetch dependencies. The required mathlib revision is `0df444a360eaa60ab8c11dca51a86af692955474`. Obtain the compiled cache for that pinned dependency set, following the official mathlib installation guidance. Do not silently substitute a newer revision, regenerate a lockfile to new versions, or edit the proof sources to make an unexplained build failure disappear.

The project also needs Python 3 and a POSIX-compatible shell. The verifier uses `flock` if present, but can run without it. There is no asserted short runtime: thousands of exact arithmetic modules can take substantial time. The script passes Lean's `-M4096` internal allocation limit per process; this is not an operating-system resident-memory cap and is not a promise the build runs on a machine with only 4 GiB total RAM. Record CPU, RAM, operating system, dependency revisions, start and finish times if reporting an independent run.

4 Run the local-source replay

From the extracted archive root, execute

```
bash verify.sh
```

The script first verifies the source hashes in the manifest. It obtains the selected Lean executable and dependency search path through Lake, prepends the local directory to `LEAN_PATH`, and compiles every local module in the listed dependency order. Each module is passed to Lean as source and produces a local object and a log in `verification-logs`. A nonzero compiler exit terminates the script. The final raw canonical harness is checked after the root module, and the final axiom report is validated against the standard allowed set.

A successful run must reach the concluding message

```
Canonical first-question theorem compiled with standard axioms only.
```

Do not infer completion from an empty log, the existence of an object file, or a partial progress list. Retain the entire command's exit status and final logs. For a second explicit raw-statement check after a successful replay, use the full Lake dependency path as well as the local object directory:

```
LEAN_PATH=".:$(lake env printenv LEAN_PATH)" \  
lake env lean -M4096 VERIFY_CANONICAL.lean
```

The expected main-theorem axiom report contains `propext`, `Classical.choice`, and `Quot.sound`, with no `sorryAx` or custom axiom. Different successful builds need not produce byte-identical compiled objects; source identity and successful kernel checks are the relevant reproducibility claims.

If the run fails, save the failing module name, exact source hash, compiler version, dependency revision, exit status and log. Distinguish an environment or resource failure from a rejected proof. Re-running an unchanged failed module with adequate resources is different from modifying the artifact. Report any source change as a new version, not a replay of version 5.

5 Inspect the proof in layers

1. Read the endpoint proof in `Erdos883AKEndpoints.lean`. Its odd-triple pigeonhole argument removes an external Ahlswede–Khachatrian assumption.
2. Follow smoothing, ordering and the exact criterion through `Erdos883Smoothing.lean`, `Erdos883RetentionOrder.lean`, `Erdos883SortedSignatures.lean`, `Erdos883OrderedHall.lean`, and `Erdos883FiniteCriterion.lean`.
3. Inspect the profile and resource bridges, including `Erdos883ProfileCriterion.lean` and `Erdos883RealResources.lean`. The two connector pools and one common skeleton must remain visible.
4. Inspect the exact finite arithmetic through the adaptive assembly files, their metadata/permutation proofs, checked count trees, and rank or span witnesses. The 88 adaptive intervals cover 1212 through 199999; the preceding proved prefix covers 0 through 1211.
5. Follow the refined CDF bounds and analytic branches in `Erdos883RefinedCdfBounds.lean`, `Erdos883ImprovedAccounting.lean`, `Erdos883ImprovedRanks.lean`, `Erdos883ImprovedNeighbors.lean`, and `Erdos883ImprovedLargeNCover.lean`.
6. Finish at `Erdos883VerifiedCoverage.lean` and the raw canonical harness. A theorem name alone is insufficient; inspect the actual type.

The manuscript's Appendix A and the companion `adaptive_intervals.csv` provide the adaptive interval list. Their endpoints were extracted from the final root and checked for count, first and last endpoints, and adjacency during manuscript preparation.

6 Interpret the existing evidence precisely

The delivered audit records modular kernel source compilation during construction, targeted recompilation of identified correspondence concerns, a fresh final-root check, the explicit raw-type check, a local-source safety scan, and unchanged frozen source/object/configuration hashes during that final check. This supports the stated formal result.

It is not a claim that every numerical certificate was rebuilt twice from scratch in an independent environment. Some construction-time module compilations have historical execution evidence but no durable per-module pre/post hash attestation. The package explains that distinction. A successful fresh run of Section 4 would add a clean local-source replay; this guide has not performed or certified that additional run. Neither the existing audit nor such a replay is independent human review or a separate Lean-kernel implementation.

7 Rebuild the explanatory documents

The clean editable source archive contains this guide, the manuscript, the one-page contribution comparison and `build.sh`. It needs XeLaTeX with Latin Modern and Noto Serif CJK SC fonts. Running `sh build.sh` compiles each document twice without shell escape. It does not run Lean. The manuscript has been rendered and reviewed page by page for layout; typesetting quality does not certify a mathematical theorem.

8 Complete the scholarly record before release

The existing draft byline is Yicheng Pan. Confirm the preferred byline, affiliation if any, contact address, authorship responsibility, and any coauthors or acknowledgments. Write a factual account of the author’s own contributions and review; do not infer these from the availability of a finished AI-assisted artifact. Complete the AI tool/model and usage disclosure. Do not describe automated checks as external human review.

Credit Della Pietra’s earlier asymptotic architecture directly, preserve the historical Erdős–Sárközy and Sárközy references, and distinguish first-question formalization from the separate second-question project. No first-ever or exclusive-priority claim is justified by this package. Add a stable public artifact identifier only after an authorized deposit. This preparation creates no publication, submission or expert-contact action.